PYTHON ABL

BY:
Sidhardha Dekka

Case Study 13: ATM Transaction Sequence Simulation:

1. Problem Statement

Process a sequence of deposits and withdrawals, maintain the account balance, reject invalid transactions, and print a transaction summary.

2. Objectives

Use variables, lists, loops, conditional statements, and arithmetic operators to simulate basic ATM transactions.

3. Assumptions

The opening balance is non-negative. Transactions are given as tuples in the form (type, amount). A withdrawal is accepted only when the amount is positive and does not exceed the balance.

4. Algorithm

1. Set the opening balance and transaction list.

2. Visit each transaction using a loop.

3. If the amount is not positive, reject it.

4. For a deposit, add the amount to the balance.

5. For a withdrawal, subtract only if enough money is available; otherwise reject it.

6. Print the final balance and accepted/rejected counts.

5. Flowchart (text version)

START → Set opening balance and transactions → Process next transaction → Is amount positive? → No: reject → Yes: deposit or withdrawal? → Check balance for withdrawal → Update balance if valid → Repeat until all transactions processed → Print summary → END

6. Pseudocode

START

balance ← opening balance

accepted ← 0; rejected ← 0

FOR each (type, amount) in transactions

IF amount <= 0: reject; rejected ← rejected + 1

ELSE IF type = 'D': balance ← balance + amount; accepted ← accepted + 1

ELSE IF type = 'W' AND amount <= balance: balance ← balance - amount; accepted ← accepted + 1

ELSE: reject; rejected ← rejected + 1

PRINT balance, accepted, rejected

END

7. Python Source Code

```
balance = 1000

transactions = [
    ("D", 500),
    ("W", 200),
    ("W", 2000),
    ("D", 0),
    ("W", 100)
]


accepted = 0

rejected = 0


for transaction_type, amount in transactions:
    if amount <= 0:
        print("Rejected: amount must be positive")
        rejected += 1
    elif transaction_type == "D":
        balance += amount
        print("Deposit accepted:", amount)
```

```
    accepted += 1
  elif transaction_type == "W":
    if amount <= balance:
      balance -= amount
      print("Withdrawal accepted:", amount)
      accepted += 1
    else:
      print("Rejected: insufficient balance")
      rejected += 1
  else:
    print("Rejected: invalid transaction type")
    rejected += 1


print("Final balance:", balance)
print("Accepted transactions:", accepted)
print("Rejected transactions:", rejected)
```

8. Test Case(s)

Opening balance = 1000; transactions = D 500, W 200, W 2000, D 0, W 100. Expected final balance = 1200; accepted = 3; rejected = 2.

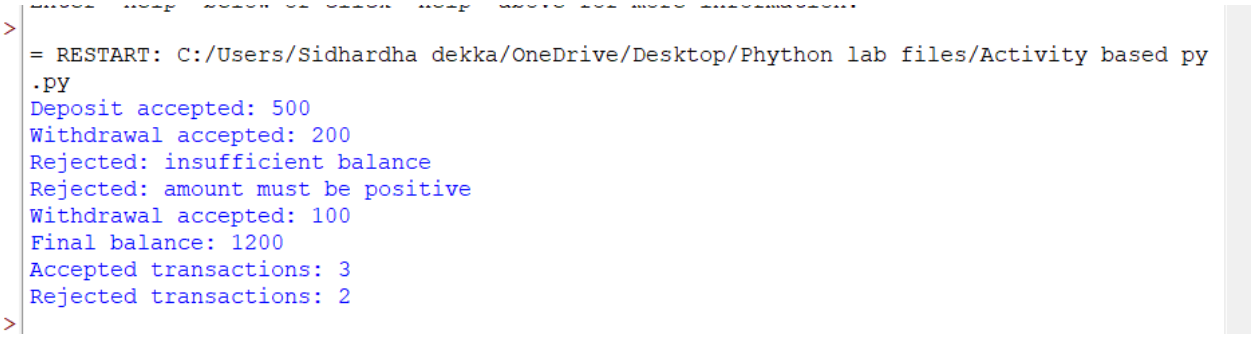
Sample Output

```
Deposit accepted: 500
Withdrawal accepted: 200
Rejected: insufficient balance
Rejected: amount must be positive
Withdrawal accepted: 100
Final balance: 1200
```

Accepted transactions: 3

Rejected transactions: 2

9. Output Screenshot



10. Conclusion

The program updates the balance only for valid transactions and produces a simple transaction summary.

Case Study 27: Advanced Numeric Pattern Generator:

1. Problem Statement

Generate a symmetric numeric diamond pattern for a user-supplied positive size and reject invalid sizes.

2. Objectives

Practice input, integer conversion, nested loops, conditions, and pattern generation.

3. Assumptions

The size must be an integer from 1 to 9 so the pattern remains small and easy to read.

4. Algorithm

1. Read the size n.

2. If n is outside 1 to 9, show an error.

3. Print the top half using increasing row numbers.

4. Print the bottom half using decreasing row numbers.

5. Use spaces to align the pattern.

5. Flowchart (text version)

START → Input n → Is 1 ≤ n ≤ 9? → No: print error → Yes: loop rows 1 to n → print spaces and increasing numbers → loop rows n-1 down to 1 → print spaces and numbers → END

6. Pseudocode

```
START
INPUT n
IF n < 1 OR n > 9: PRINT error
ELSE
  FOR i from 1 to n
    PRINT (n-i) spaces and numbers 1 to i
  FOR i from n-1 down to 1
    PRINT (n-i) spaces and numbers 1 to i
```

END

7. Python Source Code

```
n = int(input("Enter pattern size (1 to 9): "))


if n < 1 or n > 9:
   print("Invalid size. Enter a number from 1 to 9.")
else:
   for i in range(1, n + 1):
      print(" " * (n - i), end="")
      for j in range(1, i + 1):
         print(j, end=" ")
      print()


   for i in range(n - 1, 0, -1):
      print(" " * (n - i), end="")
      for j in range(1, i + 1):
         print(j, end=" ")
      print()
```

8. Test Case(s)

Input 4 → pattern has rows increasing from 1 to 4 and then decreasing to 1. Input 0 → invalid-size message.

Sample Output

```
Enter pattern size (1 to 9): 4
   1
  1 2
 1 2 3
```

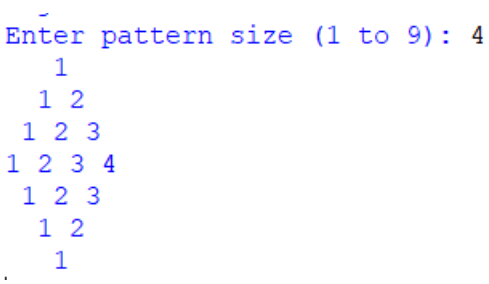
1 2 3 4

1 2 3

1 2

1

9. Output Screenshot



10. Conclusion

Nested loops can control both the number of rows and the values printed in each row.

Case Study 29: Password Policy Validation Engine:

1. Problem Statement

Check a sample password against simple rules and report each rule that it fails.

2. Objectives

Use strings, loops, Boolean variables, and conditional statements to validate input.

3. Assumptions

A valid password must contain at least 8 characters, one uppercase letter, one lowercase letter, one digit, and no spaces.

4. Algorithm

1. Read the password.

2. Check its length.

3. Loop through characters and identify uppercase, lowercase, and digits.

4. Check whether the password contains a space.

5. Print each failed rule or report that all rules passed.

5. Flowchart (text version)

START → Input password → Check length and characters → Check for spaces → Any rule failed? → Yes: print failed rules → No: print valid password → END

6. Pseudocode

START

INPUT password

Create empty failed_rules list

IF length < 8: add length rule

Set upper, lower, digit to False

FOR each character: update upper/lower/digit flags

IF upper is False: add uppercase rule

IF lower is False: add lowercase rule

IF digit is False: add digit rule

IF space exists: add no-spaces rule

PRINT result

END

7. Python Source Code

```
password = input("Enter a sample password: ")
failed_rules = []

if len(password) < 8:
  failed_rules.append("At least 8 characters")

has_upper = False
has_lower = False
has_digit = False

for ch in password:
  if ch.isupper():
    has_upper = True
  if ch.islower():
    has_lower = True
  if ch.isdigit():
    has_digit = True

if not has_upper:
  failed_rules.append("At least one uppercase letter")
if not has_lower:
```

```
    failed_rules.append("At least one lowercase letter")
if not has_digit:
    failed_rules.append("At least one digit")
if " " in password:
    failed_rules.append("No spaces allowed")


if len(failed_rules) == 0:
    print("Password meets all sample rules.")
else:
    print("Password does not meet the rules:")
    for rule in failed_rules:
        print("-", rule)
```

8. Test Case(s)

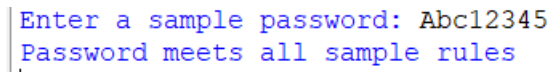
Input Abc12345 → all rules pass. Input abc 12 → fails length, uppercase, and no-spaces rules.

Sample Output

Enter a sample password: Abc12345

Password meets all sample rules.

9. Output Screenshot



10. Conclusion

The program checks a password using basic string operations. These are classroom rules, not a complete real-world security system.

Case Study 35: Text Compression Using Character Runs:

1. Problem Statement

Compress a string by recording each character followed by the number of consecutive times it appears, and compare lengths.

2. Objectives

Practice strings, loops, indexing, and conditional statements.

3. Assumptions

The input is a non-empty string. Compression is shown only when the encoded result is shorter than the original.

4. Algorithm

1. Read the text.

2. Set the first character as current and count it.

3. Loop through the remaining characters.

4. If the character matches the current character, increase the count.

5. Otherwise, add the previous character and count to the result and reset the count.

6. Append the final run and compare lengths.

5. Flowchart (text version)

START → Input text → Set current character and count → Process next character → Same character? → Increase count / save old run and reset → Save final run → Compare lengths → Print result → END

6. Pseudocode

START

INPUT text

IF text is empty: print message

ELSE set encoded = ''; current = first character; count = 1

FOR each remaining character

IF character = current: count ← count + 1

ELSE append current and count; update current and count

Append final current and count

IF length(encoded) < length(text): print encoded

ELSE print original is shorter or equal

END

7. Python Source Code

```
text = input("Enter text to compress: ")


if text == "":
   print("Please enter some text.")
else:
   encoded = ""
   current = text[0]
   count = 1


   for ch in text[1:]:
      if ch == current:
         count += 1
      else:
         encoded += current + str(count)
         current = ch
         count = 1


   encoded += current + str(count)


   print("Original text:", text)
```

```
print("Encoded text:", encoded)
print("Original length:", len(text))
print("Encoded length:", len(encoded))

if len(encoded) < len(text):
    print("Compression is beneficial.")
else:
    print("Compression is not beneficial for this text.")
```

8. Test Case(s)

Input aaabbc → encoded a3b2c1; original length 6, encoded length 6, so compression is not beneficial. Input aaaaaa → encoded a6; encoded length 2.

Sample Output

Enter text to compress: aaaaaa

Original text: aaaaaa

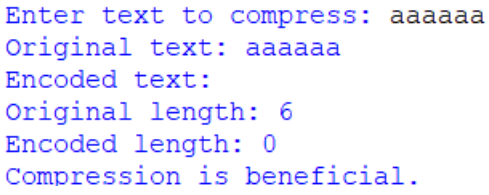
Encoded text: a6

Original length: 6

Encoded length: 2

Compression is beneficial.

9. Output Screenshot



10. Conclusion

Consecutive repeated characters can be represented by a character and its count. The length comparison shows when this simple method helps.

Case Study 37: Nested Loop Calendar Generator:

1. Problem Statement

Display a monthly calendar using the first weekday and the number of days in the month.

2. Objectives

Use nested loops, arithmetic, input validation, and formatted output.

3. Assumptions

Weekday is entered as 0 for Monday through 6 for Sunday. Number of days must be from 28 to 31. This program does not determine leap years automatically.

4. Algorithm

1. Read number of days and starting weekday.

2. Validate both inputs.

3. Print weekday headings.

4. Print blank spaces for days before the first weekday.

5. Print each date and start a new row after Sunday.

5. Flowchart (text version)

START → Input days and starting weekday → Validate values → Print headings → Print leading spaces → Loop through dates → Print date → End row every 7 columns → END

6. Pseudocode

START

INPUT days, start_day

IF days not in 28..31 OR start_day not in 0..6: print error

ELSE print weekday headings

PRINT start_day blank cells

FOR date from 1 to days

PRINT date in fixed-width cell

IF (start_day + date) MOD 7 = 0: move to next line

END

7. Python Source Code

```
days = int(input("Enter days in month (28 to 31): "))
start_day = int(input("First weekday (0=Mon, 1=Tue, ..., 6=Sun): "))


if days < 28 or days > 31 or start_day < 0 or start_day > 6:
  print("Invalid input.")
else:
  print("Mon Tue Wed Thu Fri Sat Sun")
  print("    " * start_day, end="")


  for date in range(1, days + 1):
    print(f"{date:>3}", end=" ")
    if (start_day + date) % 7 == 0:
      print()
  print()
```

8. Test Case(s)

Input days=30 and start_day=2 → first date appears under Wednesday. Input start_day=7 → invalid input.

Sample Output

```
Enter days in month (28 to 31): 30
First weekday (0=Mon, 1=Tue, ..., 6=Sun): 2
Mon Tue Wed Thu Fri Sat Sun
      1  2  3  4  5
 6  7  8  9 10 11 12
13 14 15 16 17 18 19
```

20 21 22 23 24 25 26

27 28 29 30

9. Output Screenshot

```
Enter days in month (28 to 31): 30
First weekday (0=Mon, 1=Tue, ..., 6=Sun): 2
Mon Tue Wed Thu Fri Sat Sun
          1   2   3   4   5
  6   7   8   9  10  11  12
 13  14  15  16  17  18  19
 20  21  22  23  24  25  26
 27  28  29  30
```

10. Conclusion

The starting weekday determines the spaces before the first date, while a loop prints all dates.

Case Study 42: Student Marks Matrix Analysis:

1. Problem Statement

Analyse a matrix of marks for multiple students and subjects, calculate totals and averages, and identify the top-performing student.

2. Objectives

Practice nested lists, nested loops, totals, averages, and comparisons.

3. Assumptions

Each row represents one student and each column represents one subject. Marks are between 0 and 100.

4. Algorithm

1. Store student names and marks in a nested list.

2. For each student, calculate total and average.

3. Print each student's results.

4. Track the highest total and the corresponding student.

5. Calculate each subject's average.

5. Flowchart (text version)

START → Create names and marks matrix → Loop through students → Calculate row total and average → Print student result → Track top total → Loop through subject columns → Calculate averages → Print summary → END

6. Pseudocode

START

Store names and marks matrix

top_index ← 0

FOR each student row

total ← sum of row; average ← total / number of subjects

PRINT name, total, average

IF total > current top total: update top_index

FOR each subject column

Calculate and print subject average

PRINT top student

END

7. Python Source Code

```
names = ["Asha", "Balu", "Charan"]
marks = [
  [80, 75, 90],
  [65, 70, 72],
  [92, 88, 95]
]

top_index = 0

for i in range(len(marks)):
  total = sum(marks[i])
  average = total / len(marks[i])
  print(names[i], "Total:", total, "Average:", round(average, 2))

  if total > sum(marks[top_index]):
    top_index = i

print("Top student:", names[top_index])

subject_count = len(marks[0])
for subject in range(subject_count):
```

```
  total = 0
  for student in range(len(marks)):
    total += marks[student][subject]
  average = total / len(marks)
  print("Subject", subject + 1, "average:", round(average, 2))
```

8. Test Case(s)

Asha=[80,75,90], Balu=[65,70,72], Charan=[92,88,95]. Expected top student = Charan; totals = 245, 207, 275.

Sample Output

Asha Total: 245 Average: 81.67

Balu Total: 207 Average: 69.0

Charan Total: 275 Average: 91.67

Top student: Charan

Subject 1 average: 79.0

Subject 2 average: 77.67

Subject 3 average: 85.67

9. Output Screenshot

```
Asha Total: 245 Average: 81.67
Balu Total: 207 Average: 69.0
Charan Total: 275 Average: 91.67
Top student: Charan
Subject 1 average: 79.0
Subject 2 average: 77.67
Subject 3 average: 85.67
```

10. Conclusion

Nested lists store tabular marks, and loops calculate student-wise and subject-wise statistics.

Case Study 45: Weekly Electricity Consumption Matrix:

1. Problem Statement

Store appliance-wise electricity consumption for seven days, calculate daily and appliance totals, and flag daily usage above a limit.

2. Objectives

Use nested lists, loops, sums, and conditions to analyse a small matrix.

3. Assumptions

Values are sample kWh readings. The daily limit in this example is 12 kWh.

4. Algorithm

1. Create a list of appliance names and a matrix of seven daily readings for each appliance.

2. Calculate each appliance's weekly total.

3. For each day, add readings from all appliances.

4. Print daily total and flag totals above the limit.

5. Flowchart (text version)

START → Create appliance matrix → Calculate each appliance weekly total → For each day sum appliance readings → Compare with limit → Print total and warning if needed → END

6. Pseudocode

```
START

Store appliance names and 7-day readings

FOR each appliance row: print sum(row)

FOR day from 0 to 6

 daily_total ← 0

 FOR each appliance: daily_total ← daily_total + reading[appliance][day]

 PRINT daily_total

 IF daily_total > limit: PRINT warning
```

END

7. Python Source Code

```
appliances = ["Fan", "Light", "Computer"]

# Each row contains Monday to Sunday readings in kWh.

readings = [

  [2, 2, 2, 2, 2, 3, 3],

  [1, 1, 1, 1, 1, 1, 1],

  [5, 6, 5, 7, 6, 8, 7]

]

daily_limit = 12


for i in range(len(appliances)):

  print(appliances[i], "weekly usage:", sum(readings[i]), "kWh")


for day in range(7):

  daily_total = 0

  for appliance in range(len(readings)):

    daily_total += readings[appliance][day]


  print("Day", day + 1, "total:", daily_total, "kWh")

  if daily_total > daily_limit:

    print("Warning: usage exceeded the daily limit.")
```

8. Test Case(s)

Monday total = 2+1+5=8 kWh. Saturday total = 3+1+8=12 kWh, which does not exceed the limit. Any total above 12 should trigger a warning.

Sample Output

Fan weekly usage: 16 kWh

Light weekly usage: 7 kWh

Computer weekly usage: 44 kWh

Day 1 total: 8 kWh

Day 2 total: 9 kWh

Day 3 total: 8 kWh

Day 4 total: 10 kWh

Day 5 total: 9 kWh

Day 6 total: 12 kWh

Day 7 total: 11 kWh

9. Output Screenshot

```
Fan weekly usage: 16 kWh
Light weekly usage: 7 kWh
Computer weekly usage: 44 kWh
Day 1 total: 8 kWh
Day 2 total: 9 kWh
Day 3 total: 8 kWh
Day 4 total: 10 kWh
Day 5 total: 9 kWh
Day 6 total: 12 kWh
Day 7 total: 11 kWh
```

10. Conclusion

A nested list makes it easy to store appliance readings and compare daily totals with a limit.

Case Study 49: Tuple-Based Product Catalogue Audit:

1. Problem Statement

Store product IDs, prices, and stock quantities in tuples; identify invalid records, calculate stock value, and flag low stock.

2. Objectives

Use tuples, tuple unpacking, loops, and conditions.

3. Assumptions

Each tuple is (product_id, price, quantity). Price must be non-negative and quantity must be a non-negative integer. Reorder when stock is below 5.

4. Algorithm

1. Store product records as tuples.

2. Unpack each tuple into ID, price, and quantity.

3. Check validity.

4. For valid records, calculate price multiplied by quantity.

5. Flag records with stock below 5.

6. Print total inventory value.

5. Flowchart (text version)

START → Read product tuples → Unpack one tuple → Check price and quantity → Invalid? report → Valid: calculate stock value → Check low stock → Repeat → Print total inventory value → END

6. Pseudocode

```
START

Store product tuples

inventory_value ← 0

FOR each (id, price, quantity)

  IF price < 0 OR quantity < 0: print invalid record

  ELSE inventory_value ← inventory_value + price * quantity
```

IF quantity < 5: print reorder warning

PRINT inventory_value

END

7. Python Source Code

```
products = [
  ("P101", 50.0, 10),
  ("P102", 120.0, 3),
  ("P103", 25.0, 0),
  ("P104", -10.0, 4)
]

total_value = 0

for product_id, price, quantity in products:
  if price < 0 or quantity < 0:
    print(product_id, "has an invalid record.")
  else:
    value = price * quantity
    total_value += value
    print(product_id, "stock value:", value)

    if quantity < 5:
      print("Reorder needed for", product_id)

print("Total valid inventory value:", total_value)
```

8. Test Case(s)

P101 value=500; P102 value=360 and low stock; P103 value=0 and low stock; P104 invalid due to negative price. Total valid inventory value = 860.

Sample Output

P101 stock value: 500.0

P102 stock value: 360.0

Reorder needed for P102

P103 stock value: 0.0

Reorder needed for P103

P104 has an invalid record.

Total valid inventory value: 860.0

9. Output Screenshot

```
P101 stock value: 500.0
P102 stock value: 360.0
Reorder needed for P102
P103 stock value: 0.0
Reorder needed for P103
P104 has an invalid record.
Total valid inventory value: 860.0
```

10. Conclusion

Tuples can represent fixed product records, and tuple unpacking makes each field easy to check.

Case Study 53: Food Waste Monitoring System:

1. Problem Statement

Compare meals prepared, meals served, and leftovers over several days; calculate wastage percentages and flag days above a limit.

2. Objectives

Practice lists, loops, arithmetic, and conditional statements.

3. Assumptions

Leftovers = prepared meals – served meals. Prepared meals must be greater than zero, and served meals cannot exceed prepared meals. Wastage percentage is leftovers divided by prepared meals times 100. The warning threshold is 15%.

4. Algorithm

1. Store daily prepared and served meal counts.

2. For each day, validate the counts.

3. Calculate leftovers and wastage percentage.

4. Print the results.

5. Flag any day whose wastage percentage exceeds 15%.

5. Flowchart (text version)

START → Store daily counts → Validate counts → Calculate leftovers and percentage → Print results → Percentage > 15? → Yes: warning → Repeat for all days → END

6. Pseudocode

```
START

Store prepared and served lists

FOR each day

 IF prepared <= 0 OR served < 0 OR served > prepared: print invalid data

 ELSE leftovers ← prepared - served

    waste_percent ← leftovers / prepared * 100

    PRINT leftovers and waste_percent
```

IF waste_percent > 15: PRINT warning

END

7. Python Source Code

```
days = ["Monday", "Tuesday", "Wednesday", "Thursday"]
prepared = [100, 120, 90, 110]
served = [90, 100, 80, 105]
limit = 15


for i in range(len(days)):
  if prepared[i] <= 0 or served[i] < 0 or served[i] > prepared[i]:
    print(days[i], "has invalid data.")
  else:
    leftovers = prepared[i] - served[i]
    waste_percent = leftovers / prepared[i] * 100


    print(days[i], "leftovers:", leftovers,
        "Wastage:", round(waste_percent, 2), "%")


    if waste_percent > limit:
      print("Warning: wastage is above 15%.")
```

8. Test Case(s)

Monday: prepared=100, served=90 → 10 leftovers, 10%. Tuesday: 120 prepared, 100 served → 20 leftovers, 16.67%, warning.

Sample Output

Monday leftovers: 10 Wastage: 10.0 %

Tuesday leftovers: 20 Wastage: 16.67 %

Warning: wastage is above 15%.

Wednesday leftovers: 10 Wastage: 11.11 %

Thursday leftovers: 5 Wastage: 4.55 %

9. Output Screenshot

```
Monday leftovers: 10 Wastage: 10.0 %
Tuesday leftovers: 20 Wastage: 16.67 %
Warning: wastage is above 15%.
Wednesday leftovers: 10 Wastage: 11.11 %
Thursday leftovers: 5 Wastage: 4.55 %
```

10. Conclusion

The program measures food waste as a percentage and helps identify days when waste is higher than the chosen limit.

Case Study 63: Multi-Product Price Fluctuation Analysis:

1. Problem Statement

Analyse historical prices of several products, calculate percentage change, and flag products whose change exceeds a chosen threshold.

2. Objectives

Use lists, loops, arithmetic, and conditions to compare first and last prices.

3. Assumptions

Each product has a positive starting price. Percentage change is calculated from the first price to the last price. A change greater than 10% in either direction is flagged using absolute percentage change.

4. Algorithm

1. Store product names and their price lists.

2. For each product, take the first and last prices.

3. Calculate percentage change.

4. Print the change.

5. Flag products whose absolute percentage change is greater than 10%.

5. Flowchart (text version)

START → Store products and prices → Read first and last price → Calculate percentage change → Print result → Absolute change > threshold? → Flag if yes → Repeat → END

6. Pseudocode

```
START

Store names and price lists

FOR each product

 first ← first price; last ← last price

 change ← (last - first) / first * 100

 PRINT change

 IF absolute(change) > 10: PRINT fluctuation warning
```

END

7. Python Source Code

```
products = ["Rice", "Milk", "Oil"]
prices = [
  [50, 52, 55],
  [30, 31, 32],
  [100, 95, 85]
]
threshold = 10


for i in range(len(products)):
  first_price = prices[i][0]
  last_price = prices[i][-1]


  if first_price <= 0:
    print(products[i], "has an invalid starting price.")
  else:
    change = (last_price - first_price) / first_price * 100
    print(products[i], "percentage change:", round(change, 2), "%")


    if abs(change) > threshold:
      print("Warning: price fluctuation exceeds 10%.")
```

8. Test Case(s)

Rice: 50 to 55 → +10%. Milk: 30 to 32 → +6.67%. Oil: 100 to 85 → −15%, so it is flagged. Exactly 10% is not flagged because the rule is greater than 10%.

Sample Output

Rice percentage change: 10.0 %

Milk percentage change: 6.67 %

Oil percentage change: -15.0 %

Warning: price fluctuation exceeds 10%.

9. Output Screenshot

```
Rice percentage change: 10.0 %
Milk percentage change: 6.67 %
Oil percentage change: -15.0 %
Warning: price fluctuation exceeds 10%.
```

10. Conclusion

Comparing the first and last recorded prices gives a simple measure of price movement over the supplied period.